

Embeddings

You’ve learned that text is converted to tokens, and each has a unique identifier called a token ID. But that’s just a number. It doesn’t say anything about what it means.

It’s the same as the number assigned to your Student ID. It might let you in the building, but it doesn’t tell anyone whether you are funny, into hockey, or the person who steals fries at lunch.



AI has a way to turn those token IDs into meaning. It’s called **Embedding**.

To see how this works, you and your friends decide to rate Coke and a cup of coffee on six characteristics: Sweet, Bitter, Fizz, Heat, Caffeine, and Dark.

Here are your results. We added a column for Token ID, even though that wasn’t part of your test.

Taste Profile · Coke vs. Coffee

TOKEN	TOKEN ID	DIMENSIONS					
		SWEET	BITTER	FIZZ	HEAT	CAFFEINE	DARK
 Coke	24317	9	1	10	2	3	8
 Coffee	51820	1	9	0	9	8	10

If someone asked you, “Which drink has sweet of 9, bitter of 1, and fizz of 10?” you’d immediately answer Coke.

By rating each drink on those dimensions, you’ve done what AI does on a massive scale: you’ve represented a word’s meaning with a row of numbers.

A row of numbers in a specific order, like this, is called a **Vector**. Each slot (Sweet, Bitter, etc.) is a **Dimension**. And each number is a **Value**.

WHAT ABOUT PEPSI?

Now add a third drink to the taste test: Pepsi. Score it on the same six dimensions and a problem shows up. Pepsi looks almost exactly like Coke: both sweet, both fizzy, both lightly caffeinated, neither bitter nor hot. On these six numbers alone, you cannot tell them apart.

Taste Profile · Coke vs. Pepsi vs. Coffee

TOKEN	TOKEN ID	DIMENSIONS						
		SWEET	BITTER	FIZZ	HEAT	CAFFEINE	DARK	CITRUS
 Coke	24317	9	1	10	2	3	8	1
 Pepsi	38106	9	1	10	2	3	8	10
 Coffee	51820	1	9	0	9	8	10	0

To separate them, you added a new dimension, **Citrus**. Pepsi scores high on it while Coke sits near zero, and only then do the two rows finally differ. Different numbers, different meanings: the rows no longer just name two drinks, they tell them apart.

But AI can’t add a unique dimension like Citrus for every difference between all the words in the English language. So instead of a handful of labeled dimensions, real models give each token **thousands of dimensions**, learned during training.

Here’s an important point, easy to miss: every token in the vocabulary is scored on the same dimensions. In the taste test, that would mean scoring unrelated words like “map” and “truck” on Sweet and Fizz too (they’d sit near zero, and that’s fine). What changes from token to token is the values, never the dimensions. That’s what makes any two rows comparable at all.

INSIDE A REAL MODEL

Here’s how it all fits together. The word **cat** is tokenized and assigned the **token ID** of 9246, which looks up a row in the **embedding table** (a giant lookup of every token’s numbers). That row is cat’s **embedding vector**: its full profile of values across the dimensions. The numbers that fill the table, plus many more throughout the model, are called **parameters**. Notice the dimensions aren’t named like in your taste test: during training, the model decides what each one tracks, and we usually can’t tell.

Inside a real model

1 · THE WORD

cat

2 · TOKEN ID

9246

3 · EMBEDDING TABLE

Look up row 9246

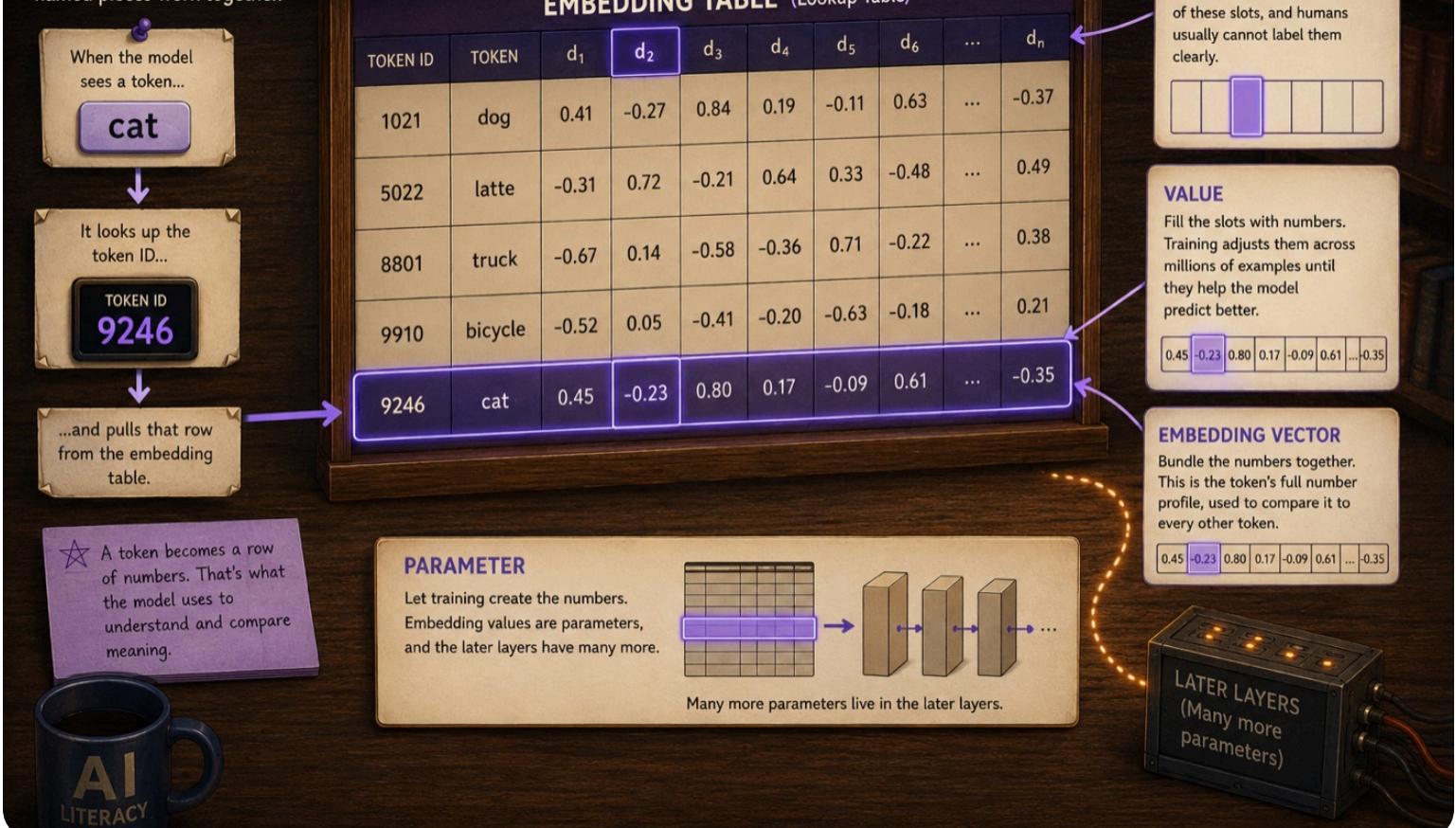
4 · EMBEDDING VECTOR

[0.71, −0.34, 1.28, 0.05, −1.62, ...]

The numbers filling that table, plus many more in the layers, are the model’s parameters.

Inside a real AI model

Follow one token to see how the five named pieces work together.



DOES EVERY TOKEN GET ITS OWN VECTOR?

Yes. Remember how ‘unbelievable’ became three tokens? Each one (‘un’, ‘believ’, ‘able’) gets its own vector with thousands of values. The model does not start with the meaning of the full word. Combining those pieces into the meaning of the whole word happens later, in the layers.

Meaning is a row of numbers.

Same dimensions for every token. Only the values change.